

LifeDump MVP Specification

Overview

LifeDump is a personal-use application that allows users to quickly capture unstructured thoughts, tasks, financial transactions, journals, and notes.

The application uses AI to automatically categorize each input into:

- Task
- Financial record
- Note

Before the information is saved, the user reviews and confirms the AI categorization.

MVP Features

1. Universal input box
 2. Text input
 3. Image upload
 4. Voice recording
 5. Tasks and reminders
 6. Expenses and income
 7. Journal and general notes
 8. Weekly summary
-

Core Mental Model

```
graph TD; A[User submits text, image, or voice] --> B[Create a temporary dump]; B --> C[Extract text or transcript]; C --> D[ ];
```

```
graph TD; A[AI categorizes the content] --> B[AI returns one or multiple structured items]; B --> C[Show confirmation screen]; C --> D[User edits, removes, or changes categories]; D --> E[User confirms the categorization]; E --> F[Save items into their respective categories]; F --> G[Generate a weekly summary from the saved data];
```

One user input may contain multiple categories.

Example Input

Remind me to submit my assignment tomorrow, I spent Rp45,000 on lunch, and today I felt productive.

Expected Output

1. Task: Submit assignment tomorrow
2. Expense: Lunch, Rp45,000
3. Note: Felt productive today

Technical Stack

- Next.js
- Firebase
- Clerk
- Zustand
- TanStack React Query
- Vercel AI SDK
- OpenRouter API
- shadcn/ui

Design Direction

The application should use a simple, neutral shadcn/ui design.

Design Principles

- Mobile-first
- Neutral colors
- Minimal use of decorative colors
- Strong typography hierarchy
- Generous spacing
- Thin borders
- Large touch targets
- One-column mobile layout
- Icons combined with text labels
- Simple and calm interface

The application should feel like a personal notebook rather than a complex productivity dashboard.

Data Model

The application should separate the original user submission from the categorized results.

1. Dump

A dump represents the original content submitted by the user.

```
type DumpSourceType = "text" | "image" | "voice";

type DumpStatus =
  | "processing"
  | "needs_review"
  | "confirmed"
  | "failed";

type Dump = {
  id: string;
  userId: string;

  sourceType: DumpSourceType;

  rawText?: string;
  transcript?: string;
  mediaPath?: string;

  status: DumpStatus;

  createdAt: Date;
```

```
    updatedAt: Date;
};
```

The original dump should be saved before AI processing begins. This prevents user content from being lost when an upload, transcription, or AI request fails.

2. Item

An item represents an individual result extracted from a dump.

```
type ItemCategory = "task" | "financial" | "note";

type Item = {
  id: string;
  userId: string;
  dumpId: string;

  category: ItemCategory;

  title: string;
  content?: string;

  task?: {
    dueAt?: Date;
    isCompleted: boolean;
  };

  financial?: {
    type: "expense" | "income";
    amount: number;
    currency: "IDR";
    occurredAt: Date;
  };

  note?: {
    noteType: "journal" | "general";
  };

  aiConfidence?: number;

  createdAt: Date;
  updatedAt: Date;
};
```

One dump may generate multiple items.

AI Categorization

The AI should return structured data instead of unstructured text.

Use a Zod schema with the AI SDK to validate the model output.

Example AI Response

```
{
  "items": [
    {
      "category": "task",
      "title": "Submit accounting assignment",
      "content": null,
      "dueAt": "2026-06-17T23:59:00+07:00",
      "confidence": 0.94,
      "needsClarification": false
    },
    {
      "category": "financial",
      "title": "Lunch",
      "financialType": "expense",
      "amount": 45000,
      "currency": "IDR",
      "occurredAt": "2026-06-16",
      "confidence": 0.98,
      "needsClarification": false
    }
  ],
  "assumptions": [
    "Tomorrow was interpreted using the Asia/Jakarta timezone."
  ]
}
```

AI Rules

The AI must:

- Never save data automatically.
- Never invent missing amounts.

- Never invent missing dates.
- Return `null` when information is unavailable.
- Support one input producing multiple items.
- Recognize Indonesian and English expressions.
- Recognize Indonesian date expressions such as:
 - Besok
 - Lusa
 - Minggu depan
 - Nanti malam
- Recognize common Indonesian currency expressions such as:
 - Rp50.000
 - 50rb
 - 50k
 - Lima puluh ribu
- Use `Asia/Jakarta` as the default timezone.
- Use IDR as the default currency.
- Include a confidence score.
- Explain important assumptions.

The OpenRouter API key must only be used on the server.

Confirmation Screen

The confirmation screen is a core part of the MVP.

The user must approve the AI-generated items before they are permanently saved.

Example

TASK

Submit accounting assignment

Due:

Tomorrow, 11:59 PM

[Edit] [Change Category] [Remove]

EXPENSE

Lunch

Amount:

Rp45,000

Date:

Today

[Edit] [Change Category] [Remove]

[Cancel] [Confirm All]

User Actions

The user should be able to:

- Edit the title.
- Edit the content.
- Change the category.
- Change the due date.
- Change the amount.
- Change expense into income.
- Remove an incorrectly extracted item.
- Add another item manually.
- Confirm all items.
- Cancel the dump.

For mobile devices, use a full-page review screen or a shadow **Drawer** instead of a small dialog.

Input Processing

Text Input

User enters text

↓

Create dump

↓

Send text to AI

↓

Receive categorized items

↓

Show confirmation screen

Text should be implemented first because it is the simplest input format.

Voice Input

```
Record audio
  ↓
Upload audio
  ↓
Transcribe audio
  ↓
Categorize transcript
  ↓
Show confirmation screen
```

Voice MVP Requirements

- Maximum recording duration of approximately 60 seconds
 - One recording per dump
 - Show the transcript before confirmation
 - Allow users to edit the transcript
 - Preserve audio when processing fails
 - Delete audio when the dump is permanently deleted
-

Image Input

```
Upload image
  ↓
Store image
  ↓
Vision model extracts content
  ↓
Categorize extracted content
  ↓
Show confirmation screen
```

Initial Image Support

- Screenshots
- Receipt photos
- Photos of handwritten notes
- Photos of typed documents

Set limits for:

- Maximum file size

- Accepted image formats
 - Image dimensions
-

Tasks and Reminders

The MVP task system should remain simple.

Included Features

- Task title
- Optional description
- Optional due date
- Optional due time
- Today section
- Upcoming section
- Overdue section
- Completed section
- Mark task as completed
- Reopen completed task
- Edit task
- Delete task

Excluded From the Initial MVP

- Recurring tasks
- Subtasks
- Calendar synchronization
- Task priorities
- Projects
- Tags
- Browser push notifications

For the first version, reminders may be represented through in-app due-date indicators.

Push notifications can be added later.

Financial Records

The financial section should support both expenses and income.

Financial Item Fields

```
type FinancialItem = {  
  type: "expense" | "income";  
  title: string;  
  amount: number;  
  currency: "IDR";  
  occurredAt: Date;  
  note?: string;  
};
```

Included Features

- Add expense
- Add income
- Edit transaction
- Delete transaction
- Transaction history
- Weekly income total
- Weekly expense total
- Weekly net balance

Excluded From the Initial MVP

- Budgets
- Financial goals
- Bank synchronization
- Multiple currencies
- Automatic bank transaction imports
- Advanced financial charts
- Recurring transactions

Financial totals should always be calculated by application code, not by AI.

Notes and Journal

Notes should be divided into two simple types:

- General note
- Journal entry

Included Features

- Create note
- Edit note
- Delete note
- View notes chronologically
- Distinguish general notes from journal entries
- Search notes by title or content

Excluded From the Initial MVP

- Folders
 - Tags
 - Rich text editing
 - Mood analysis
 - Semantic search
 - AI chat with notes
 - Collaborative notes
-

Weekly Summary

The weekly summary should combine deterministic calculations with AI-generated writing.

Application-Calculated Metrics

```
type WeeklyMetrics = {  
  tasksCreated: number;  
  tasksCompleted: number;  
  tasksPending: number;  
  tasksOverdue: number;  
  
  totalExpense: number;  
  totalIncome: number;  
  netBalance: number;  
  
  notesCreated: number;  
  journalEntriesCreated: number;  
};
```

The application should calculate all totals before sending the summary data to the AI.

The AI should only turn the verified metrics into a readable narrative.

Example Summary

You completed 8 of the 12 tasks you created this week.

Your total expenses were Rp735,000, while your total income was Rp1,500,000.

You also created six notes. University assignments and organizational activities appeared frequently in your entries.

Weekly Summary Flow

```
graph TD; A[Collect weekly data] --> B[Calculate task and financial metrics in code]; B --> C[Select relevant note information]; C --> D[Send verified data to AI]; D --> E[Generate readable summary]; E --> F[Save generated summary];
```

Weekly Summary ID

Use a stable identifier such as:

2026-W25

Generate the summary when the user opens it for the first time.

After generation, cache the summary in Firebase.

When data from that week changes, mark the summary as stale and provide a [Regenerate Summary](#) button.

Firestore Structure

```
users/{userId}
  dumps/{dumpId}
  items/{itemId}
  weeklySummaries/{weekId}
```

Dump Document Example

```
{
  sourceType: "voice",
  transcript: "I spent fifty thousand rupiah on dinner",
  mediaPath: "users/userId/dumps/dumpId/audio.webm",
  status: "needs_review",
  createdAt: serverTimestamp(),
  updatedAt: serverTimestamp()
}
```

Item Document Example

```
{
  dumpId: "dump_123",
  category: "financial",
  title: "Dinner",
  financial: {
    type: "expense",
    amount: 50000,
    currency: "IDR",
    occurredAt: Timestamp
  },
  createdAt: serverTimestamp(),
  updatedAt: serverTimestamp()
}
```

Use a Firestore batched write when the user confirms multiple items.

The confirmation should either save all selected items successfully or save none of them.

Authentication and Security

Clerk should be the main authentication provider.

Firebase should recognize the authenticated Clerk user through a secure token integration.

Security Requirements

- Each user can only access their own data.
- Never trust a `userId` sent directly from the browser.
- Validate the authenticated user on the server.
- Protect Firestore documents with security rules.
- Protect Firebase Storage files with security rules.
- Keep OpenRouter API keys on the server.
- Keep Firebase Admin credentials on the server.
- Do not store sensitive secrets in public environment variables.

Data Paths

```
users/{authenticatedUserId}/dumps/{dumpId}
users/{authenticatedUserId}/items/{itemId}
users/{authenticatedUserId}/weeklySummaries/{weekId}
```

Personal journals, voice recordings, and financial records should be treated as sensitive data.

Library Responsibilities

Next.js

Use Next.js App Router for:

- Pages
- Layouts
- Server-side authentication
- API route handlers
- AI requests
- Firebase Admin operations
- Protected server logic

Suggested structure:

```
app/  
  api/  
    categorize/  
    transcribe/  
    weekly-summary/  
  
  home/  
  tasks/  
  money/  
  notes/
```

React Query

Use React Query for server state:

- Fetching tasks
- Fetching financial records
- Fetching notes
- Fetching dumps
- Creating items
- Updating items
- Deleting items
- Loading weekly summaries
- Invalidating cached queries

Example query keys:

```
["items", userId]  
["tasks", userId]  
["financial", userId]  
["notes", userId]  
["weekly-summary", userId, weekId]
```

Zustand

Use Zustand only for temporary client-side state:

- Current input text
- Selected image
- Voice recording state
- Audio preview

- Extracted AI items
- Confirmation-screen edits
- Current review step
- Open or closed drawer state

Do not duplicate all Firebase data inside Zustand.

React Query should own remote data. Zustand should own temporary interface state.

AI SDK

Use the AI SDK for:

- Structured AI output
- Zod schema validation
- AI provider integration
- Categorization
- Weekly-summary generation

All AI requests must be performed server-side.

Firebase

Use Firebase for:

- Firestore database
- Firebase Storage
- Server timestamps
- Batched writes
- User-specific data storage

Firebase Storage should store:

- Voice recordings
 - Uploaded images
 - Receipt photos
-

Mobile-First Navigation

Use four primary bottom-navigation items:

Home

- Main input box
- Text input
- Camera or image-upload button
- Voice-recording button
- Recent dumps
- Weekly-summary card

Tasks

- Today
- Upcoming
- Overdue
- Completed

Money

- Weekly income
- Weekly expenses
- Net balance
- Transaction history

Notes

- General notes
- Journal entries
- Note history

The weekly summary should appear on the Home page instead of having its own navigation tab.

Suggested Home Screen

LifeDump

What's on your mind?

Type anything here...

[Image] [Voice] [Dump]

This Week

- 8 tasks completed
- Rp735,000 spent
- 6 notes created

Recent

- Submit accounting assignment
- Lunch – Rp45,000
- Presentation reflection

Category Indicators

Do not use color as the only way to distinguish categories.

Use icons and labels.

- ✓ Task
- \$ Financial
- Note

Category labels should remain visible on confirmation cards and list items.

Application States

The interface should explicitly support the following states:

Idle
Uploading
Transcribing
Analyzing
Needs review
Confirmed
AI failed
Upload failed
No items detected

Failure Recovery

When processing fails, the app should preserve the original dump.

The user should be able to:

- Try again
- Edit the original input
- Categorize manually
- Delete the dump

The user should not need to re-enter or re-record content because an AI request failed.

Recommended Development Order

Phase 1: Foundation

- Set up Next.js
- Set up shadcn/ui
- Set up Clerk
- Set up Firebase
- Create protected routes
- Create Firestore security rules
- Create basic mobile layout

Phase 2: Text Input

- Build text input
- Create dump records
- Add AI categorization
- Add structured output validation
- Build confirmation screen
- Save confirmed items

Phase 3: Category Pages

- Build Tasks page
- Build Money page
- Build Notes page
- Add edit and delete functionality
- Add React Query caching

Phase 4: Weekly Summary

- Calculate weekly metrics
- Generate AI narrative
- Cache weekly summary
- Add regenerate functionality

Phase 5: Voice Input

- Record audio
- Upload audio
- Transcribe audio
- Allow transcript editing
- Send transcript through categorization pipeline

Phase 6: Image Input

- Upload image
- Extract image content
- Handle receipt images and screenshots
- Send extracted content through categorization pipeline

Features Excluded From the MVP

The following features should not be included initially:

- Budgets
- Financial goals
- Recurring tasks
- Recurring transactions
- Subtasks
- Task priorities
- Tags
- Folders
- Mood tracking
- Mood analysis
- Calendar synchronization
- Push notifications
- Collaboration
- Chat with personal data
- Semantic search
- Advanced analytics
- Advanced charts
- Multiple currencies
- Offline synchronization

- Bank-account integration
- Automatic AI advice

These features may be considered after the core workflow has been validated.

MVP Completion Criteria

The MVP is complete when a user can:

1. Sign in securely.
 2. Submit text, an image, or a voice recording.
 3. Have the input analyzed by AI.
 4. Receive one or multiple categorized items.
 5. Review and correct the AI output.
 6. Confirm the categorized items.
 7. View tasks, financial records, and notes separately.
 8. Complete and reopen tasks.
 9. Edit and delete saved items.
 10. View an accurate weekly summary.
 11. Access only their own private data.
 12. Recover their original input when AI processing fails.
-

Main Product Hypothesis

The MVP should test the following hypothesis:

Users will consistently submit unstructured personal information when they do not need to organize it manually, as long as they can review and correct the AI categorization before it is saved.

Core Product Principle

You do not need to organize your thoughts before entering them. LifeDump organizes them for you.